

Besoins non fonctionnels du CRM

Document détaillé de révision et de préparation à la soutenance

1. Définition générale

Les besoins non fonctionnels décrivent les qualités attendues du CRM et les contraintes auxquelles il doit répondre. Ils répondent principalement à la question : « Comment le système doit-il fonctionner ? » Ils concernent notamment la sécurité, la scalabilité, la maintenabilité, la performance, la disponibilité, la fiabilité, l'intégrité des données, la traçabilité et l'ergonomie.

2. Synthèse

Besoin	Explication	Application CRM
Sécurité	Protéger comptes, API et données.	Authentification, rôles, permissions et contrôle d'accès.
Scalabilité	Supporter l'augmentation des utilisateurs et données.	Séparation Flutter / Backend / PostgreSQL et architecture modulaire.
Maintenabilité	Faciliter corrections et évolutions.	Modules, responsabilités séparées et migrations.
Performance	Conserver des temps de réponse acceptables.	Pagination, filtres et optimisation SQL/API.
Disponibilité	Maintenir le service accessible.	Supervision, redémarrage, sauvegardes et infrastructure adaptée.
Fiabilité	Garantir des traitements cohérents.	Validations, règles métier et gestion des erreurs.
Intégrité	Empêcher les données incohérentes.	Contraintes PostgreSQL, relations et validations backend.
Traçabilité	Identifier qui fait quoi et quand.	Historique, utilisateur, timestamps et logs selon le besoin.
Ergonomie	Faciliter le travail des utilisateurs.	Navigation, formulaires, tableaux, filtres et dashboards.
Responsive	Adapter l'interface aux écrans.	PC, tablette et smartphone.

3. Sécurité

Définition : La sécurité protège les utilisateurs, les données et les fonctionnalités contre les accès non autorisés.

Application : Dans le CRM, l'authentification doit être complétée par l'autorisation : chaque rôle ne doit accéder qu'aux modules et actions qui lui sont permis.

Question du jury : Jury : « Quelle différence entre authentification et autorisation ? » Réponse : l'authentification vérifie l'identité ; l'autorisation détermine les droits.

4. Scalabilité

Définition : La scalabilité est la capacité à évoluer lorsque le nombre d'utilisateurs, de données ou de requêtes augmente.

Application : La séparation Flutter → API Backend → PostgreSQL permet d'optimiser les couches indépendamment. Une scalabilité à grande échelle doit toutefois être validée par des tests de charge.

Question du jury : Jury : « Votre CRM est-il scalable ? » Réponse : l'architecture favorise la scalabilité ; une garantie réelle nécessite mesures et tests de charge.

5. Maintenabilité

Définition : La maintenabilité correspond à la facilité de corriger, modifier et faire évoluer le logiciel.

Application : La modularisation et la séparation des responsabilités limitent l'impact d'une modification sur les autres modules.

Question du jury : Jury : « Pourquoi la modularité aide-t-elle ? » Réponse : elle réduit le couplage et facilite les changements.

6. Performance

Définition : La performance concerne le temps de réponse et l'utilisation efficace des ressources.

Application : Pagination, filtres, requêtes SQL optimisées et limitation des données transférées permettent de maîtriser les volumes.

Question du jury : Jury : « Que faire si les données augmentent fortement ? » Réponse : indexation adaptée, pagination, optimisation SQL, cache si pertinent et tests de charge.

7. Disponibilité

Définition : La disponibilité signifie que le CRM reste accessible lorsque les utilisateurs en ont besoin.

Application : Elle dépend du frontend, du backend, de PostgreSQL et de l'infrastructure. Supervision, redémarrage et sauvegardes réduisent les risques.

Question du jury : Jury : « Que se passe-t-il si le backend tombe ? » Réponse : supervision, logs, redémarrage et stratégie de récupération doivent être prévus.

8. Fiabilité

Définition : La fiabilité garantit des résultats cohérents et prévisibles.

Application : Validation frontend + validation backend + règles métier + gestion des exceptions, avec transactions lorsque nécessaire.

Question du jury : Jury : « Pourquoi valider aussi côté backend ? » Réponse : le frontend n'est pas une frontière de sécurité.

9. Intégrité des données

Définition : L'intégrité garantit que les données respectent les relations et contraintes définies.

Application : PostgreSQL protège l'intégrité structurelle avec clés et contraintes ; le backend applique les règles métier.

Question du jury : Jury : « Pourquoi des contraintes en base si le backend valide ? » Réponse : la base fournit une protection supplémentaire contre les incohérences.

10. Traçabilité

Définition : La traçabilité permet de retrouver qui a effectué une opération, sur quelle ressource et à quel moment.

Application : Historique, utilisateur associé, timestamps, statuts et logs peuvent être utilisés selon les opérations à auditer.

Question du jury : Jury : « Comment savoir qui a validé ? » Réponse : conserver l'identité de l'utilisateur et les informations temporelles dans le mécanisme d'audit.

11. Ergonomie

Définition : L'ergonomie rend les tâches métier simples et compréhensibles.

Application : Navigation cohérente, formulaires, filtres, tableaux, dashboards, messages d'erreur et confirmations réduisent les erreurs.

Question du jury : Jury : « Pourquoi est-ce non fonctionnel ? » Réponse : cela décrit une qualité d'utilisation du système plutôt qu'une fonction métier précise.

12. Responsive et compatibilité

Définition : Le CRM doit rester utilisable sur PC, tablette et téléphone.

Application : Les menus, tableaux, formulaires et boutons doivent s'adapter à la largeur disponible et aux interactions tactiles.

Question du jury : Jury : « Pourquoi ne pas faire trois interfaces ? » Réponse : le responsive permet de mutualiser les composants tout en adaptant leur présentation.

13. Autres exigences à considérer

Extensibilité : permettre l'ajout de nouveaux modules et règles métier.

Testabilité : faciliter les tests unitaires, API et fonctionnels.

Robustesse : gérer les erreurs, entrées invalides et situations inattendues.

Interopérabilité : permettre les échanges avec d'autres services via API et intégrations.

Sauvegarde et récupération : permettre la restauration des données après incident.

Évolutivité fonctionnelle : faire évoluer les workflows sans remettre en cause toute l'architecture.

Portabilité : permettre l'utilisation dans les environnements ciblés avec un minimum d'adaptations.

14. Méthode de réponse devant le jury

Pour chaque besoin : 1) donner sa définition ; 2) expliquer pourquoi il est important pour le CRM ; 3) montrer la réponse architecturale ou technique ; 4) indiquer comment il peut être mesuré ou vérifié ; 5) reconnaître les limites et proposer les améliorations possibles.

15. Formulation générale pour la soutenance

« Les besoins non fonctionnels définissent les qualités attendues de notre CRM. Nous avons particulièrement pris en compte la sécurité, la maintenabilité, la scalabilité, la performance, la fiabilité, l'intégrité des données, la disponibilité, la traçabilité et l'ergonomie. Ces exigences influencent directement l'architecture, l'API, la base de données et l'interface utilisateur. »